



LEX / FLEX

Section - 1

What is LEX/Flex ?

- ▶ Lexical analyzer (scanner) breaks up input stream into **valid tokens**.
- ▶ LEX is a tool to generate lexical analyzers.

- ▶ **Why use LEX?**
 - ↳ Automates lexical analysis.
 - ↳ Uses regular expressions.
 - ↳ Easy integration with YACC.

Working of LEX/Flex



- ▶ Write LEX specification (.l file)
- ▶ Run lex/flex command (**flex sample.l**)
- ▶ Compile lex.yy.c (**gcc lex.yy.c**)
- ▶ Execute (**./a.exe**)

Structure of LEX Program

- ▶ LEX program has sections as below:

%{

// Declaration (C language)

%}

Definition (optional)

%%

Rules (pattern action)

%%

User code section (subroutines)

Structure of LEX Program

► Declaration

→ Syntax:

```
%{  
    // Declaration (C language)  
}%
```

→ Sample:

```
%{  
#include <stdio.h>  
  
int count = 0;  
}%
```

Structure of LEX Program

► Definition

↳ Syntax:

Definition (optional)

%%

↳ Sample:

Letter [a-zA-Z]

Digit [0-9]

EOL \n

%%

Structure of LEX Program

► Rules

→ Syntax:

Rules (pattern|action)

%% ↓
 space

→ Sample:

[0-9] {printf("Numbers: %s",yytext);}

Letter {count++; printf("%s",yytext);}

. {}

%%

Structure of LEX Program

► User Subroutines

→ Syntax:

User code section (subroutines in C language)

→ Sample:

```
void main() {  
    yylex();  
    printf("Number of letters are %d",count);  
}  
  
int yywrap() {  
    return 1;  
}
```


Regular Expression operators

a	Matches character itself. abc → matches abc itself
.	Any character except new line. a. → matches a followed by any character
[]	Matches one character from []. [abc] → matches either a or b or c
[^]	Matches any character not in []. [^a] → matches any character not a
*	Zero or more occurrences. a* → matches null, a, aa, aaa, ...
+	one or more occurrences. a+ → matches a, aa, aaa, ...
?	Matches zero or one occurrence. a? → matches null or a
()	Groups expressions together. (ab)+ → matches ab, abab, ababab, ...
^	Matches beginning of a line. ^int → matches line beginning with int
\$	Matches end of a line. ;\$ → matches line ending with ;
\	Escapes special characters. \. → matches .
-	Range. [a-z] → matches a to z

Regular Expression operators

|

Matches Logical OR. $a|c \rightarrow$ matches a or c

“ ”

Treats characters inside as literals. $."a*b" \rightarrow$ matches exactly a*b

Inbuilt Functions and variables

- ▶ **yytext** : char * Pointer to the current matched lexeme
- ▶ **yytext** : int Length of the matched lexeme
- ▶ **yyin** : FILE * Input file pointer
- ▶ **yyout** : FILE* Output file pointer
- ▶ **yylex()** : Main lexical analyzer function. Scans input and matches pattern written in rule section. Pattern will be matched in sequence. If first pattern is not matched then next pattern will be matched. (Called by main())
- ▶ **yywrap()** or **%option noyywrap** : Called at EOF, returns 1 to stop scanning. (End of input handling)

Sample lex program

```
%option noyywrap

%{

#include <stdio.h>

int c=0,v=0;

}%

%%

[aeiouAEIOU]    {v++;}

[a-zA-Z]        {c++;}

\n              {printf("No of vowels: %d\nNo of consonants: %d\n",v,c);v=0;c=0;}

.               {}

%%

void main(){
    yylex();}
```